Ryan Stoddard

Feb. 24th, 2026

Foundations of Python Programming

Assignment 05

Dictionaries and Exception Handling

Introduction

In this document I will describe the steps I took to complete Assignment 05. This assignment asks us to create a python program that displays a menu. Depending on the user's choice within the menu they will be allowed to perform different tasks such as recording data, displaying data, and writing data to a .json file. This exercise is nearly identical to assignment 4 except that it introduces importing a module, the use of dictionaries and the try/except method for exception handling.

Loading Modules, Setting up the Constants, and Variables

Once again, we are building upon the structure of the previous assignment. Starting out with a script header. In order to gain access to features like json.load and json.dumps we need to first import the json module. Tradition demands that we import any required modules immediately after the script header. And we do this by simply typing in "import json".

Next we define the constants and variables. A new addition to this section of the code are the constant Enrollments.json and the variable student_data: dict. Instead of working with a list of lists, we will be working with a list of dictionaries.

Loading Data from an Existing .JSON File

Now that the initial setup is complete, we can load in the data from the .json file. The code to accomplish this from a .json into a Python dictionary variable is actually quite simple compared to the list we used last time. We still start with file = open(FILE_NAME, "r"), but instead of using a for loop and readlines all we need is students = json.load(file). The module we imported earlier is doing a lot of the heavy lifting here. But also, the notation for a python dictionary is pretty much identical to the format of the .json file.

In this assignment I also added some exception handling to the reading of the external data. Using the try/except/finally method we can section off certain parts of our code to run, see if an exception is found and print messages as needed. The code that occurs after the "finally" statement will run if an exception is found or not. This means that we can put the file.close() statement after the "finally" statement and it will apply no matter what logic path is taken. Adding an if statement here to check if the file is currently open first is also a good way to do clean up code. In theory, the "finally" statement used in this way makes revising/maintaining large blocks of code easier. But for a program this simple having file.close() in multiple places wouldn't significantly affect the program's performance or maintainability. In this section of the code I checked for a "FileNotFoundError" and a catch all exception (Figure 1).

```python
try:
    # Extract the data from the file
    file = open(FILE_NAME, "r")
    students = json.load(file)

# Exception for file not found
except FileNotFoundError as e:
    print("File not found, please try again")
    print("--- Technical Error Message ---")
    print(e, e.__doc__, type(e), sep="n")

# Catch all exception
except Exception as e:
    print("There was a non-specific error!\n")
    print("--- Technical Error Message ---")
    print(e, e.__doc__, type(e), sep="n")

# Check if a file object exists and is still open
finally:
    if file is not None and file.closed == False:
        file.close()
```

Figure 1: Example of Code to Load Data with Exception Handling

Menu Option 1: Inputting New Data

The next section of code that has some differences is how we add new user data to our list of dictionaries. The main difference between doing this with dictionaries versus with lists is that we use keys and values instead of just the input values. This will become useful later when we are presenting our list of dictionaries "students" to the user. Instead of using indexes we can call out keys. Keys can be called whatever we want, which makes it a lot easier to read instead of having to remember what order the desired values were listed in to call them with an index (Figure 2).

```python
        course_name = input("Please enter the name of the course: ")
        student_data = {"FirstName": student_first_name,
                        "LastName": student_last_name,
                        "CourseName": course_name}
        students.append(student_data)
        print(f"You have registered {student_first_name} {student_last_name} \
for {course_name}.")
```

Figure 2: Saving Input as a List of Dictionaries

Another difference in this section of code is the exception handling I've added. Similar to the lab from the reading, I added a check to the name inputs to ensure there are no numbers in the name, as well as a general catch all. (Figure 3)

```python
        try:
            student_first_name = input("Enter the student's first name: ")
            if not student_first_name.isalpha():
                raise ValueError("The first name should not contain numbers.")
            student_last_name = input("Enter the student's last name: ")
            if not student_last_name.isalpha():
                raise ValueError("The last name should not contain numbers.")
        except Exception as e:
            print(e)
            print("-- Technical Error Message --")
            print(e.__doc__)
            print(e.__str__)
            continue
```

Figure 3: Example of Exception Handling for User Inputs

Menu Option 2: Show Current Data

As mentioned earlier, this section of code is altered to use a list of dictionaries instead of a list of list when presenting the information to the user. Instead of using indexes to call out specific sections in our custom message we use keys. (Figure 4).

```python
    elif menu_choice == "2":

        # Process the data to create and display a custom message
        print("-"*50)
        for student in students:
            print(f"Student {student["FirstName"]} {student["LastName"]} is \
enrolled in {student["CourseName"]}")
        print("-"*50)
        continue
```

Figure 4: Example of Menu option 2

Menu Option 3: Saving Data

The next elif statement we set up runs the open/write/close sequence from previous assignment, only this time we are using the json.dump functionality from the json module. And setting up a custom print statement just like the one from Menu Option 2.

We are also setting up exception handling for this section similar to what we did when we were reading in the data from the external file. We encapsulate the code with try/except/finally, and check specifically for a “TypeError” exception to ensure the data is a valid JSON format. We then put in a catch all exception and “file.close()” after the “finally statement” (Figure 5).

```python
    # Save the data to a file
    elif menu_choice == "3":
        try:
            file = open(FILE_NAME, "w")
            json.dump(students, file)
        except TypeError as e:
            print("Please check that the data is a valid JSON format\n")
            print("--- Technical Error Message ---")
            print(e, e.__doc__, type(e), sep="n")
            continue
        except Exception as e:
            print("--- Technical Error Message ---")
            print(e, e.__doc__, type(e), sep="n")
            continue
        finally:
        # Check if a file object exists and is still open
            if file is not None and file.closed == False:
                file.close()

        print("The following data was saved to file!")
        for student in students:
            print(f"Student {student["FirstName"]} {student["LastName"]} is \
enrolled in {student["CourseName"]}")
        continue
```

Figure 5: Example of Menu option 3

Summary

The “Dictionaries and Exception Handling” assignment has introduced several new concepts. We have now seen how to import a module, utilize dictionaries, and set up both generic and customized exception handling. With dictionaries enhanced readability and surprisingly simpler code, I can see why they are so popular in the industry. There are likely still use cases for lists and .csv files when moving large amounts of predictably formatted files.

Being able to create your own custom exception handling has been an eye opener for me. Being proactive and spending some time thinking about how users might break my program lets us create something much more robust.